

Assignment 2: Building a Small-Scale Foundation Model from Scratch

mini-GPT | 6.85M parameters (baseline) | 10 training runs, 31M tokens in total | NVIDIA GeForce GTX 1650 | generated from the run logs

1. Model architecture and parameters

The model is a decoder-only transformer in the GPT-2 layout, implemented from PyTorch primitives in `src/model.py`. Token ids are embedded, added to a learned positional embedding, and passed through a stack of **pre-LayerNorm** blocks, each computing $x + \text{Attn}(\text{LN}(x))$ then $x + \text{MLP}(\text{LN}(x))$. Attention is written out explicitly: one linear map produces queries, keys and values for all heads; scores are scaled by $1/\sqrt{\text{head_dim}}$, masked with a lower-triangular causal mask, softmax-normalized and recombined. The MLP expands to 4d with a **GELU** activation. A final LayerNorm feeds a linear head whose weight is **tied** to the token embedding, producing logits over all 50,257 GPT-2 tokens at every position. Initialization follows GPT-2 ($N(0, 0.02)$, residual output projections scaled by $1/\sqrt{2L}$). A unit test checks causality directly: changing a future token leaves every earlier logit unchanged.

Component	Shape (baseline)	Parameters	Share
Token embedding (tied with output head)	50,257 x 128	6,432,896	94.0%
Positional embedding (learned)	128 x 128	16,384	0.2%
2 transformer blocks (4 heads, head_dim 32, MLP 512)	2 x (LN, QKV 128x384, proj, LN, MLP)	396,544	5.8%
Final LayerNorm	128 + 128	256	0.0%
Total (unique)		6,846,080	100%

Table 1. Baseline parameters. The 50,257-token vocabulary dominates a model this small: the embedding is 16x the size of both transformer blocks together, and the output projection onto it is most of the compute per token. Tying the head to the embedding avoids a second table of the same size.

2. Dataset

Training data is the Assignment 1 corpus: cleaned, exactly-deduplicated English text from Wikipedia, CC-News and FineWeb (wikipedia 36%, cc_news 30%, fineweb 34% of tokens), tokenized with the GPT-2 BPE tokenizer and packed into flat uint16 files with an end-of-text token between documents. The train split holds 333.7M tokens from 485,493 documents; the validation split, assigned per document by hash, holds 1.60M tokens from 2,418 documents that never appear in training.

Each run samples 1,536 random 1024-token pages of the train split (1.6M tokens per epoch, the same pages for every run) and cuts each page into consecutive windows of the run's sequence length; the target is the input shifted by one token. Because the page sample depends only on the seed, the sequence-length runs see identical text and differ only in how it is windowed. A shuffled DataLoader draws batches from the windows in a new seeded order each epoch. During training, validation loss is measured on a fixed 131K-token subset of the validation split; the final scores use a larger fixed subset of 524K tokens, the same for every run.

3. Training setup

Each step runs the forward pass, the mean cross-entropy of the next-token logits against the shifted targets, the backward pass, gradient-norm clipping, and an AdamW update. Batches larger than 16 sequences are split into micro-batches whose gradients accumulate before the single optimizer step (a unit test checks that this matches the full-batch gradient). Perplexity is exp of the token-weighted mean cross-entropy over the whole evaluation set. The average training loss of each epoch, validation loss and perplexity at evenly spaced points, learning rate, gradient norm and throughput are logged to `metrics.jsonl`; a resumable checkpoint (weights, optimizer moments, loss scale, position) is saved after every epoch.

Setting	Baseline value	Setting	Baseline value
Optimizer	AdamW, betas (0.9, 0.95)	Epochs	2
Peak learning rate	1e-3	Batch size	32 x 128 tokens
Schedule	5% linear warmup, cosine to 10%	Steps	768 (384 per epoch)
Weight decay	0.1 (matrices only)	Tokens seen	3.1M
Gradient clipping	global norm 1.0	Precision	float32
Dropout	0.0	Hardware	NVIDIA GeForce GTX 1650
Seed	1337	Throughput	28,106 tokens/s (median)

Table 2. Baseline training configuration. Dropout is off: every run sees each token only 2 times, so the model is far from memorizing its data (see the train/validation gap in Section 4), and dropout would only slow learning.

4. Baseline training dynamics

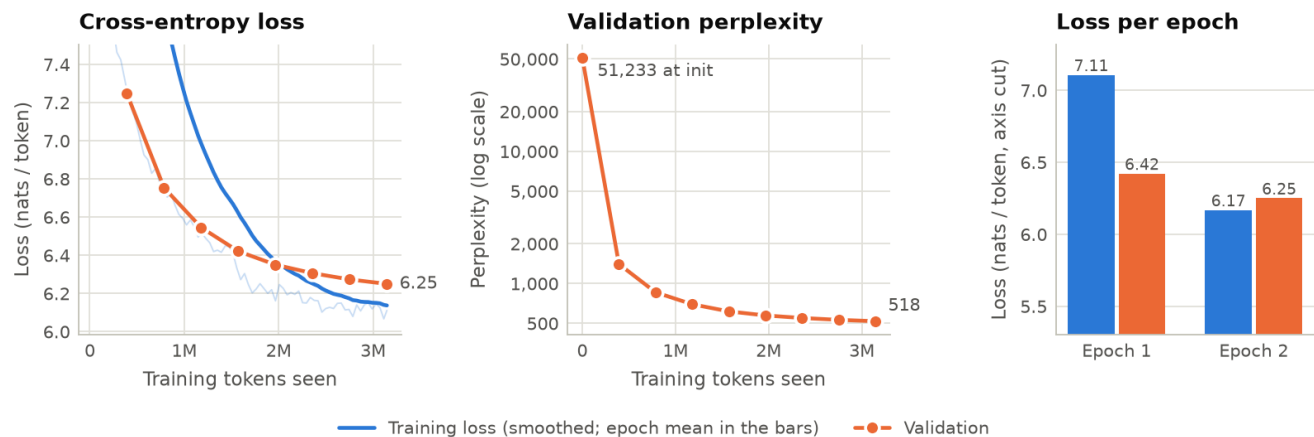


Figure 1. Baseline run. Left: training loss per 10-step window (faint) and its moving average, with validation loss. The y axis starts after the first 10% of training so the later differences are visible. Middle: validation perplexity on a log scale, from the untrained model onward. Right: the average training loss of each epoch next to the validation loss at that epoch's end.

The untrained model scores a perplexity of 51,233, close to the 50,257 a uniform guess over the vocabulary would give (loss $\ln 50,257 = 10.82$). Loss falls steeply during warmup, as the model first learns token frequencies and then common short-range patterns, and keeps falling through all 2 epochs. After training, the baseline reaches a validation loss of 6.242 on the held-out final-evaluation subset (perplexity **513.7**). In the last epoch the average training loss was 6.167 against 6.249 on validation, a gap of +0.082 nats. The two remain close, so the model is still underfitting: capacity and data, not memorization, limit it.

5. Hyperparameter experiments

The sweep changes one setting at a time from the baseline, so each difference is attributable to that setting: learning rate, batch size, number of layers, embedding size and sequence length (a full grid of the same values would be 162 runs; this is 10). Every run trains on the same pages for the same number of tokens. For batch size this is the fair comparison: at a fixed budget a larger batch takes fewer, less noisy steps, and that trade-off is what is being measured.

Run	Changed setting	Params	Steps	Val loss	Val ppl	vs base	Tokens/s
baseline	none	6.85M	768	6.242	513.7		28,106
lr_5e-4	learning rate 1e-3 -> 5e-4	6.85M	768	6.483	654.2	+27.3%	28,099
lr_3e-3	learning rate 1e-3 -> 3e-3	6.85M	768	6.247	516.6	+0.6%	28,083
batch_16	batch size 32 -> 16	6.85M	1,536	6.176	481.2	-6.3%	47,743
batch_64	batch size 32 -> 64	6.85M	384	6.453	634.3	+23.5%	24,168
layers_1	layers 2 -> 1	6.65M	768	6.295	541.6	+5.4%	39,604
embd_64	embedding size 128 -> 64	3.32M	768	6.541	692.8	+34.9%	40,770
embd_256	embedding size 128 -> 256	14.48M	768	5.994	400.8	-22.0%	14,972
seq_32	sequence length 128 -> 32	6.83M	3,072	6.109	449.7	-12.5%	29,020
seq_64	sequence length 128 -> 64	6.84M	1,536	6.127	457.9	-10.9%	27,592

Table 3. All runs, scored on the same held-out validation tokens after training. "vs base" is the change in perplexity relative to the baseline (negative is better). Bold: the best run (embd_256), exported as mini_gpt_checkpoint.pt.

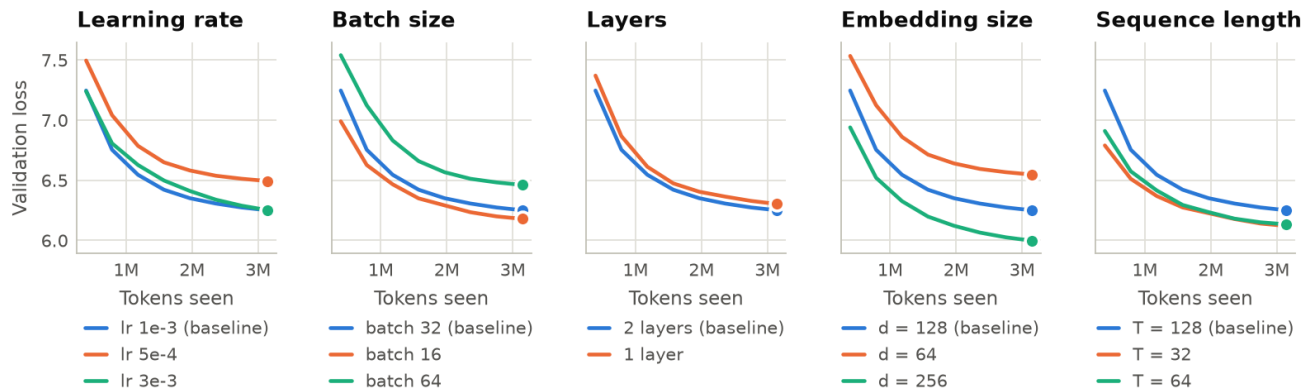


Figure 2. Validation loss during training for each factor, baseline in blue in every panel. All panels share one y axis, zoomed past the first 10% of training.

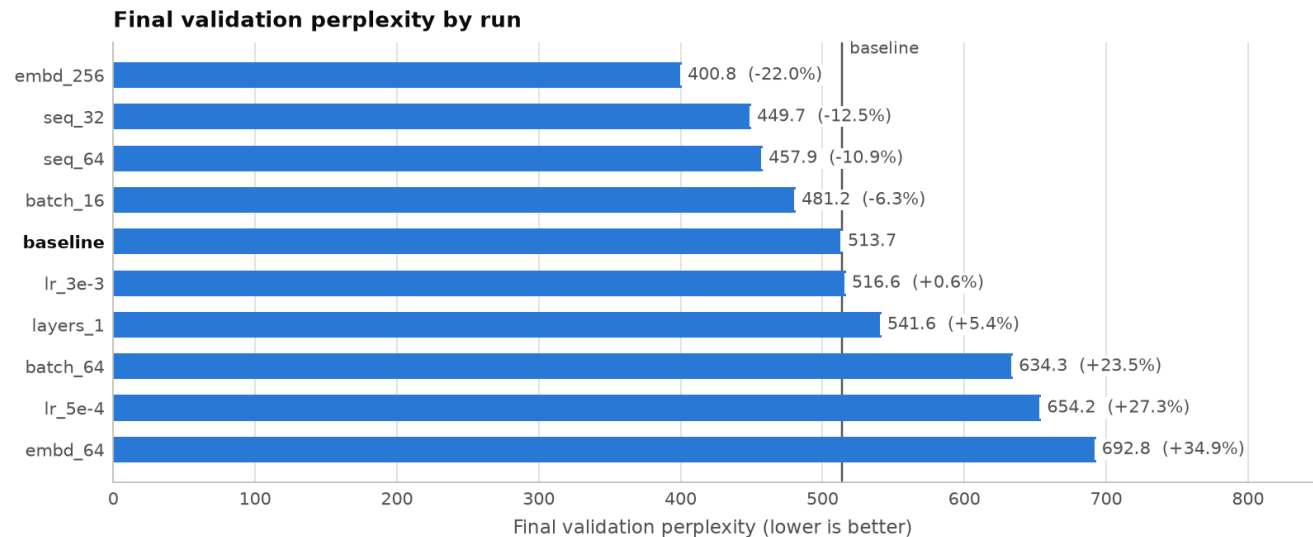


Figure 3. Final validation perplexity for every run, best first. The vertical line marks the baseline.

Findings by factor.

- **Learning rate** (perplexity 5e-4: 654.2, 1e-3: 513.7, 3e-3: 516.6). Best: 1e-3; the worst setting is 27.3% higher.
- **Batch size** (perplexity 16: 481.2, 32: 513.7, 64: 634.3). Best: 16; the worst setting is 31.8% higher.

- **Layers** (perplexity 1: 541.6, 2: 513.7). Best: 2; the worst setting is 5.4% higher.
- **Embedding size** (perplexity 64: 692.8, 128: 513.7, 256: 400.8). Best: 256; the worst setting is 72.8% higher.
- **Sequence length** (perplexity 32: 449.7, 64: 457.9, 128: 513.7). Best: 32; the worst setting is 14.2% higher.

6. Samples from the exported checkpoint

Generated by `src/generate.py`, which rebuilds the model from `mini_gpt_checkpoint.pt` alone (run `emdb_256`; temperature 0.8, top-k 40, 40 new tokens). The prompt is in bold.

The history of the city. he was at a major house to play with a record of the first time where he was a single-year-old and was a year.
the final. the first time,

Scientists announced on Tuesday that will be a lot of the day. in the first, he is a big house of one of us at the game, but i have two of
the best. the game i thought you

In computer science, a member of new hartio, a british man who served for the new zealand-year-american male and the city in the
united states | colonel viet

The samples are locally fluent at best: common function words, plausible punctuation and short phrases, but no coherence beyond a few tokens. That is what a validation perplexity in the hundreds means: the model narrows each next token to a few hundred candidates, which captures word frequencies and short-range grammar but not meaning.

7. Observations and challenges

- **Best run.** `emdb_256` reached perplexity 400.8, 22.0% below the baseline's 513.7, after 3.1M training tokens.
- **Capacity costs compute.** Doubling the embedding to 256 changed perplexity from 513.7 to 400.8 but cut throughput from 28,106 to 14,972 tokens/s, because the cost of the output projection grows linearly with d . Halving it to 64 gave 692.8 at 40,770 tokens/s. With a fixed token budget, the extra quality of a wider model is paid for in wall-clock time.
- **Depth matters less than width here.** Going from 2 layers to 1 moved perplexity from 513.7 to 541.6 (+5.4%), and removes only 198K parameters: the transformer blocks are a small share of a model whose size is set by the vocabulary.
- **The vocabulary dominates compute and memory.** The GPT-2 vocabulary makes the output projection and its 50,257-way softmax most of the work per token. The logits for one batch of 32 x 128 tokens are 0.8 GB in fp32, and with the log-softmax and gradients a 4 GB GTX 1650 ran out of memory at 32 sequences per forward pass. Gradient accumulation over micro-batches of 16 keeps the optimization batch size independent of memory (peak 1.8 GB).
- **Mixed precision was slower.** fp16 autocast measured 18.5K tokens/s against 21.1K in fp32 on the GTX 1650, which has no tensor cores; the casts cost more than the half-precision arithmetic saves. Training therefore runs in fp32, with bf16 used automatically only on GPUs that support it natively.
- **Hardware and time budget.** The first environment was CPU-only (about 2,400 tokens/s); moving to the GPU gave a roughly 10x speedup. The deadline still limited each run to a few million tokens, so every model is far from converged: validation loss is still falling at the end of every curve. The rankings describe early training. A lower learning rate or a larger batch, which lose here, may catch up with a longer budget.
- **Gradient accumulation has a throughput cost.** Batches of 32 or more run as micro-batches of 16, and the batch-16 run, which needs no split, trained about 1.7x faster in tokens/s than the baseline (Table 3). The per-micro-batch overhead of the split was not investigated further within the time available; it affects speed only, not the gradients, which a unit test shows are identical to the full-batch ones.
- **Sequence length changes the task, not only the model.** A model with a 32-token context is scored on predicting each token from at most 31 predecessors, so its perplexity is not directly comparable to a 128-token model's. The runs see identical text, but a lower loss at short context means the model learned faster per token, not that it models language better.
- **What an epoch means.** The full training split holds 334M tokens, far beyond this budget, so each run trains for two epochs over a fixed random sample of it. The small train/validation gap shows that the repetition has not yet led to memorization.